DATA 260 Homework 9
Redis Caching, Threshold Economics, and Self-Improving Agents

Student: Sanjana Thummalapalli
SID4: 7801
Domain: Clinical Trials
PREFIX: s7801
Submission Branch: hw9
GitHub Repository: https://github.com/sanjana-glitch-art/data260-7801

Personal Configuration

Configuration	Value
SID4	7801
PORT_BASE	8601
PREFIX	s7801
SEED	7801
VERIFY_SEED	267801
DOMAIN_ID	1
Assigned domain	Clinical Trials
Database	MySQL 8.4
Cache	Redis Stack 7.4
Vector algorithm	HNSW
Embedding model	all-MiniLM-L6-v2
Embedding dimensions	384
Local model	qwen3:4b

Part 1: Redis-Backed Clinical-Trial View Counter

Design

The method increment_trial_views(trial_id) increments a Redis value whenever a clinical-trial record is accessed. The Redis key follows the required personal prefix: s7801:views:clinical_trial:<trial_id>

The method uses Redis INCR to perform an atomic increment. When the count is divisible by 10, the current value is written to the clinical_trials.view_count column in MySQL.

Relevant logic:

current_count = int(self.redis_client.incr(key))if current_count % VIEW_SYNC_INTERVAL == 0: with self.engine.begin() as connection: connection.execute(text(""" UPDATE clinical_trials SET view_count = :view_count WHERE id = :trial_id """), { "trial_id": trial_id, "view_count": current_count, },)

```
Student: Sanjana Thummalapalli | SID4: 7801 | DATA-260 HW9

=== Redis View Counter Implementation ===

> code\hw9_view_counter.py:91:    def increment_trial_views(
  code\hw9_view_counter.py:92:        self,
  code\hw9_view_counter.py:93:        trial_id: int,
  code\hw9_view_counter.py:94:    ) -> int:
  code\hw9_view_counter.py:95:        """Increment Redis and sync every ten views."""
  code\hw9_view_counter.py:96:
  code\hw9_view_counter.py:97:        if not self.trial_exists(trial_id):
  code\hw9_view_counter.py:98:            raise ValueError(
  code\hw9_view_counter.py:99:                "Clinical trial not found."
  code\hw9_view_counter.py:100:            )
  code\hw9_view_counter.py:101:
  code\hw9_view_counter.py:102:        key = self.view_key(trial_id)
  code\hw9_view_counter.py:103:
  code\hw9_view_counter.py:104:        if not self.redis_client.exists(key):
  code\hw9_view_counter.py:105:            persisted_count = (
  code\hw9_view_counter.py:106:                self.database_view_count(trial_id)
  code\hw9_view_counter.py:107:            )
  code\hw9_view_counter.py:108:
  code\hw9_view_counter.py:109:            self.redis_client.set(
  code\hw9_view_counter.py:110:                key,
  code\hw9_view_counter.py:111:                persisted_count,
  code\hw9_view_counter.py:112:                nx=True,
  code\hw9_view_counter.py:113:            )
  code\hw9_view_counter.py:114:
  code\hw9_view_counter.py:115:        current_count = int(
  code\hw9_view_counter.py:116:            self.redis_client.incr(key)
  code\hw9_view_counter.py:117:        )
  code\hw9_view_counter.py:118:
  code\hw9_view_counter.py:119:        if (
  code\hw9_view_counter.py:120:            current_count
  code\hw9_view_counter.py:121:            % VIEW_SYNC_INTERVAL
  code\hw9_view_counter.py:122:            == 0
  code\hw9_view_counter.py:123:        ):
  code\hw9_view_counter.py:124:            with self.engine.begin() as connection:
  code\hw9_view_counter.py:125:                connection.execute(
  code\hw9_view_counter.py:126:                    text(
  code\hw9_view_counter.py:127:                        """
  code\hw9_view_counter.py:128:                        UPDATE clinical_trials
  code\hw9_view_counter.py:129:                        SET view_count = :view_count
  code\hw9_view_counter.py:130:                        WHERE id = :trial_id
  code\hw9_view_counter.py:131:                        """
  code\hw9_view_counter.py:132:                    ),
  code\hw9_view_counter.py:133:                    {
  code\hw9_view_counter.py:134:                        "trial_id": trial_id,
  code\hw9_view_counter.py:135:                        "view_count": current_count,
  code\hw9_view_counter.py:136:                    },
  code\hw9_view_counter.py:137:                )
  code\hw9_view_counter.py:138:
  code\hw9_view_counter.py:139:            print(
```

Figure 1: Part 1 counter implementation code

Counter Demonstration: The experiment reset the Redis and database values, then accessed the same clinical trial 25 times.

Observed behavior:

Access #1: Total = 1
...
Access #10: Total = 10
Synced views to database at count 10
...
Access #20: Total = 20
Synced views to database at count 20
...
Access #25: Total = 25

Final state:

Metric	**Result**
Simulated accesses	25
Redis count	25

MySQL persisted count	20
Synchronization points	10 and 20
Result	PASS

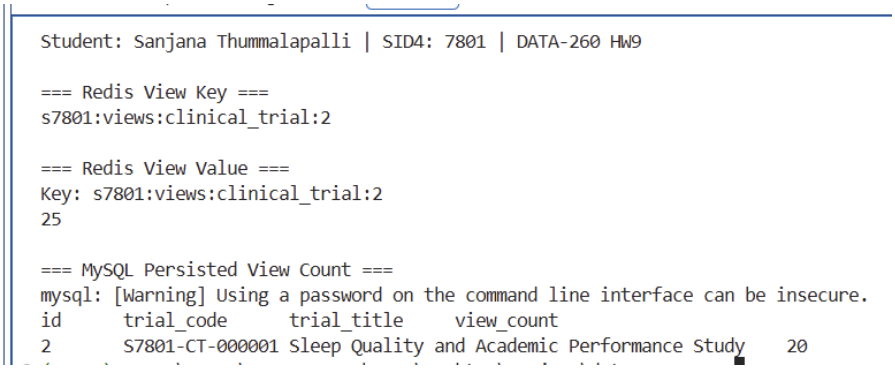
The database contains 20 rather than 25 because the design persists only at multiples of ten. Redis contains the most recent count.

```
Student: Sanjana Thummalapalli | SID4: 7801 | DATA-260 HW9

=== 25-Access Counter Demonstration ===

Access #9: Total = 9
Synced views to database at count 10
Access #10: Total = 10
Access #11: Total = 11
Access #19: Total = 19
Synced views to database at count 20
Access #20: Total = 20
Access #21: Total = 21
Access #25: Total = 25
Redis count: 25
Database count: 20
Expected database count: 20
Counter demonstration: PASS
```

Figure 2: Counter output showing accesses 10, 20 and 25



```
Student: Sanjana Thummalapalli | SID4: 7801 | DATA-260 HW9

=== Redis View Key ===
s7801:views:clinical_trial:2

=== Redis View Value ===
Key: s7801:views:clinical_trial:2
25

=== MySQL Persisted View Count ===
mysql: [Warning] Using a password on the command line interface can be insecure.
id      trial_code      trial_title     view_count
2       S7801-CT-000001 Sleep Quality and Academic Performance Study    20
```

Figure 3: Redis key and MySQL view_count

Part 2: Semantic Response Cache

Architecture

The semantic cache is placed before the existing clinical-trial agent path. Each query is embedded using all-MiniLM-L6-v2, producing a normalized 384-dimensional FLOAT32 vector.

Redis Stack stores each entry as a hash containing:

- Original query
- Embedding
- Final answer
- Data-version marker
- Creation timestamp

The Redis HNSW index is: s7801:semantic_cache:index

The index uses cosine distance. A similarity value is calculated as: similarity = 1 - cosine distance

On a cache hit, the saved response is returned without rerunning the agent. On a miss, the normal local-model agent executes and the response is stored.

```
=== Redis Vector Search ===

> code\hw9_semantic_cache.py:148:    def search(
  code\hw9_semantic_cache.py:149:        self,
  code\hw9_semantic_cache.py:150:        query: str,
  code\hw9_semantic_cache.py:151:    ) -> dict[str, Any] | None:
  code\hw9_semantic_cache.py:152:        """Return the nearest cached response."""
  code\hw9_semantic_cache.py:153:
  code\hw9_semantic_cache.py:154:        vector = self.embed(query)
  code\hw9_semantic_cache.py:155:
  code\hw9_semantic_cache.py:156:        search_query = (
  code\hw9_semantic_cache.py:157:            Query(
  code\hw9_semantic_cache.py:158:                "*=>[KNN 1 "
  code\hw9_semantic_cache.py:159:                "@embedding $query_vector "
  code\hw9_semantic_cache.py:160:                "AS vector_distance]"
  code\hw9_semantic_cache.py:161:            )
  code\hw9_semantic_cache.py:162:            .sort_by("vector_distance")
  code\hw9_semantic_cache.py:163:            .return_fields(
  code\hw9_semantic_cache.py:164:                "query",
  code\hw9_semantic_cache.py:165:                "answer",
  code\hw9_semantic_cache.py:166:                "data_version",
  code\hw9_semantic_cache.py:167:                "created_at",
  code\hw9_semantic_cache.py:168:                "vector_distance",
  code\hw9_semantic_cache.py:169:            )
  code\hw9_semantic_cache.py:170:            .dialect(2)
  code\hw9_semantic_cache.py:171:        )
  code\hw9_semantic_cache.py:172:
  code\hw9_semantic_cache.py:173:        try:
  code\hw9_semantic_cache.py:174:            result = (
  code\hw9_semantic_cache.py:175:                self.redis_client.ft(
  code\hw9_semantic_cache.py:176:                    SEMANTIC_CACHE_INDEX
  code\hw9_semantic_cache.py:177:                ).search(
  code\hw9_semantic_cache.py:178:                    search_query,
  code\hw9_semantic_cache.py:179:                    query_params={
  code\hw9_semantic_cache.py:180:                        "query_vector": (
  code\hw9_semantic_cache.py:181:                            vector.tobytes()
  code\hw9_semantic_cache.py:182:                        ),
  code\hw9_semantic_cache.py:183:                    },
  code\hw9_semantic_cache.py:184:                )
  code\hw9_semantic_cache.py:185:            )
  code\hw9_semantic_cache.py:186:        except ResponseError as error:
  code\hw9_semantic_cache.py:187:            if "No results found" in str(error):
  code\hw9_semantic_cache.py:188:                return None
  code\hw9_semantic_cache.py:189:            raise
  code\hw9_semantic_cache.py:190:
  code\hw9_semantic_cache.py:191:        if not result.docs:
  code\hw9_semantic_cache.py:192:            return None
  code\hw9_semantic_cache.py:193:
  code\hw9_semantic_cache.py:194:        document = result.docs[0]
  code\hw9_semantic_cache.py:195:
  code\hw9_semantic_cache.py:196:        distance = float(
  code\hw9_semantic_cache.py:197:            self.decode_value(
  code\hw9_semantic_cache.py:198:                document.vector_distance
  code\hw9_semantic_cache.py:199:            )
  code\hw9_semantic_cache.py:200:        )
  code\hw9_semantic_cache.py:201:
  code\hw9_semantic_cache.py:202:        similarity = 1.0 - distance
  code\hw9_semantic_cache.py:203:
  code\hw9_semantic_cache.py:204:        return {
  code\hw9_semantic_cache.py:205:            "cache_key": document.id,
  code\hw9_semantic_cache.py:206:            "cached_query": self.decode_value(
  code\hw9_semantic_cache.py:207:                document.query
  code\hw9_semantic_cache.py:208:            ),
  code\hw9_semantic_cache.py:209:            "answer": self.decode_value(
  code\hw9_semantic_cache.py:210:                document.answer
  code\hw9_semantic_cache.py:211:            ),
  code\hw9_semantic_cache.py:212:            "data_version": self.decode_value(
  code\hw9_semantic_cache.py:213:                document.data_version
```

Figure 4: Semantic-cache code showing vector search and hit/miss logic

```
Student: Sanjana Thummalapalli | SID4: 7801 | DATA-260 HW9

=== Redis Semantic Vector Index ===
Index: s7801:semantic_cache:index
Algorithm: HNSW
Data type: FLOAT32
Dimensions: 384
Distance metric: COSINE
Index available: True
```

Figure 5: Redis FT.INFO showing HNSW, FLOAT32, DIM 384 and COSINE

Ten-Query Experiment

The experiment contained exact duplicates, paraphrases and new questions.

Metric	Result
Queries	10
Cache hits	3
Cache misses	7
Cache hit rate	30.00%
Average cached latency	7.922 ms
Average non-cached latency	7105.463 ms
Starting threshold	0.85

The results show that cached queries were substantially faster. The purpose of the experiment was to record actual behavior rather than target a predetermined speed-up.

One paraphrase produced a similarity of 0.844334, slightly below the initial 0.85 threshold, and therefore correctly followed the miss path under the configured policy.

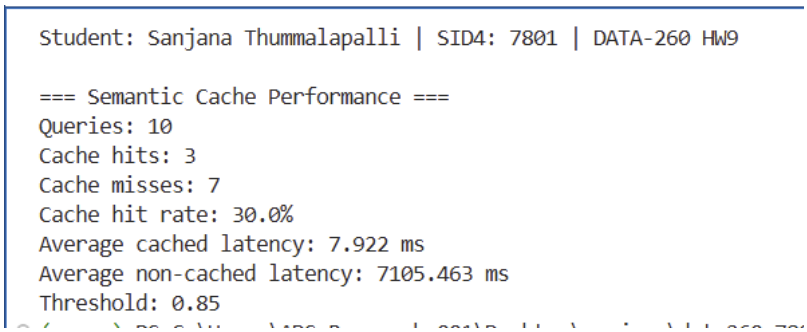


Figure 6: Ten-query cache performance summary

Part 3: Threshold Economics and Stale Answers

3.1 Labeled Test Set

Before running the threshold sweep, I created and committed cache_cases.yaml. It contains 40 cases:

Category	Cases	Expected behavior
Exact duplicates	10	Hit
Paraphrases	10	Hit
Similar wording with different answers	10	Miss
Unrelated queries	10	Miss

Total	40	20 hits, 20 misses

The labels were committed before observing threshold results and were not changed afterward.

```
Student: Sanjana Thummalapalli | SID4: 7801 | DATA-260 HW9

=== cache_cases.yaml Commit ===
2ad460b (HEAD -> hw9) Add labeled Homework 9 semantic cache cases
 cache_cases.yaml | 334 ++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++
 1 file changed, 334 insertions(+)

=== Labeled Case Validation ===
Cases: 40
Categories: {'exact_duplicate': 10, 'paraphrase': 10, 'similar_different_answer': 10, 'unrelated': 10}
Should hit: 20
Should miss: 20
```

Figure 7: Git commit and validation showing 40 balanced cases

3.2 Threshold Sweep

The 40 cases were evaluated at five thresholds.

Threshold	Hit rate	False-hit rate	Mean latency	Total cost
0.75	55.00%	30.00%	3204.516 ms	$6.36
0.80	50.00%	30.00%	3559.369 ms	$6.40
0.85	25.00%	0.00%	5335.839 ms	$0.60
0.90	25.00%	0.00%	5335.744 ms	$0.60
0.95	25.00%	0.00%	5337.649 ms	$0.60

Mean end-to-end latency uses the measured Redis lookup time and the observed Part 2 agent-miss latency.

Lower thresholds improved hit rate and latency but introduced incorrect cached responses. Thresholds of 0.85 and above eliminated false hits in this dataset.

```
Student: Sanjana Thummalapalli | SID4: 7801 | DATA-260 HW9

=== Threshold Economics ===

threshold hit_rate_percent false_hit_rate_percent mean_latency_ms total_cost
--------- ---------------- ---------------------- --------------- ----------
0.75      55.0             30.0                   3204.516        6.36
0.8       50.0             30.0                   3559.369        6.4
0.85      25.0             0.0                    5335.839        0.6
0.9       25.0             0.0                    5335.744        0.6
0.95      25.0             0.0                    5337.649        0.6


Recommended threshold: 0.9
```

Figure 8: Completed five-row threshold table

3.3 False Hit in Full

The clearest false hit occurred at threshold 0.75.

Cached query: What phase is the sleep quality trial?

Incoming query: What is the enrollment target of the sleep quality trial?

Similarity: 0.830932

Incorrect cached response: The sleep quality trial is Phase II.

Correct response: The sleep quality trial has an enrollment target of 120.

The embedding model considered the questions similar because they contain the same clinical-trial name and share most words. However, one asks for the phase while the other asks for enrollment. This demonstrates why semantic similarity alone cannot guarantee answer equivalence.

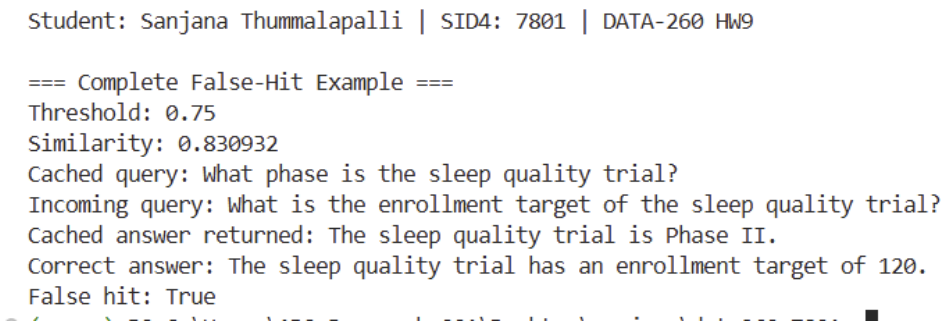


Figure 9: Full false-hit JSON

3.4 Cost Model and Threshold Choice

I assigned:

- Cache-miss cost: $0.02
- False-hit support and correction cost: $1.00

A cache miss consumes local-model compute and increases latency. A false hit is assigned a much larger cost because an incorrect clinical-trial answer reaches the user and may require support or correction.

The total cost is: total cost = number of misses × $0.02 + number of false hits × $1.00

I selected threshold 0.90 for deployment. It produced:

- Zero false hits
- The minimum total cost of $0.60
- Slightly lower mean latency than threshold 0.85

This choice favors correctness while still retaining safe exact-match cache hits.

3.5 Stale Answers and Invalidation

The cached query was: What is the enrollment target of the sleep quality trial?

Initial database value: 120

The response and data-version marker were stored in Redis. The database enrollment target was then changed to 145. Before invalidation, Redis returned the stale cached answer containing 120.

Observed stale result:

Updated database value: 145
Returned stale answer: 120
Stale hit demonstrated: True

The invalidation strategy compares each cached entry's stored data-version marker with the current relational-data version. Entries with mismatched versions are deleted.

After invalidation:

Stale entries deleted: 1
Cache miss after invalidation: True
Refreshed answer: 145
Refreshed cache correct: True

The database record was restored to its original value of 120 after the experiment. The trade-off is that version checking prevents stale answers but adds a database-version read. The current global marker may also invalidate unrelated cached answers. A record-specific version would be more precise but requires additional metadata and invalidation logic.

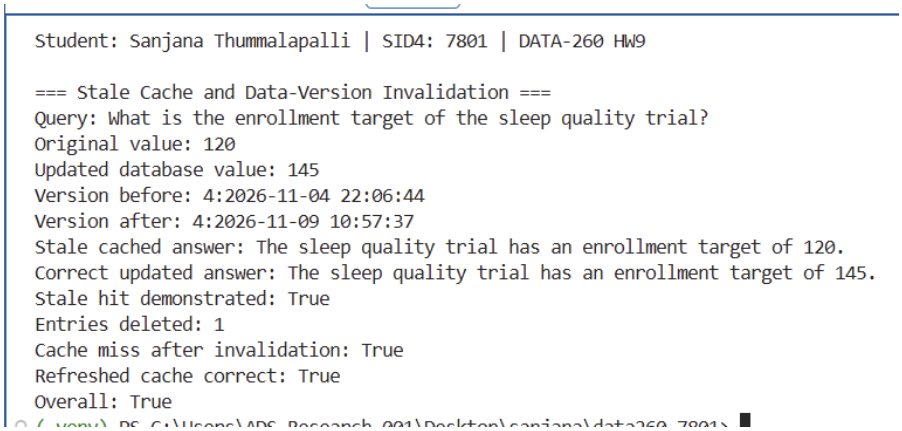


Figure 10: Stale hit before invalidation and refreshed answer after invalidation

Part 4: Self-Improving Multi-Agent System

Datasets

The system uses:

- reference_data.json: ground-truth clinical-trial facts
- training_data.json: development questions
- validation_data.json: lesson validation questions
- test_data.json: held-out final questions
- lessons.json: reusable lessons

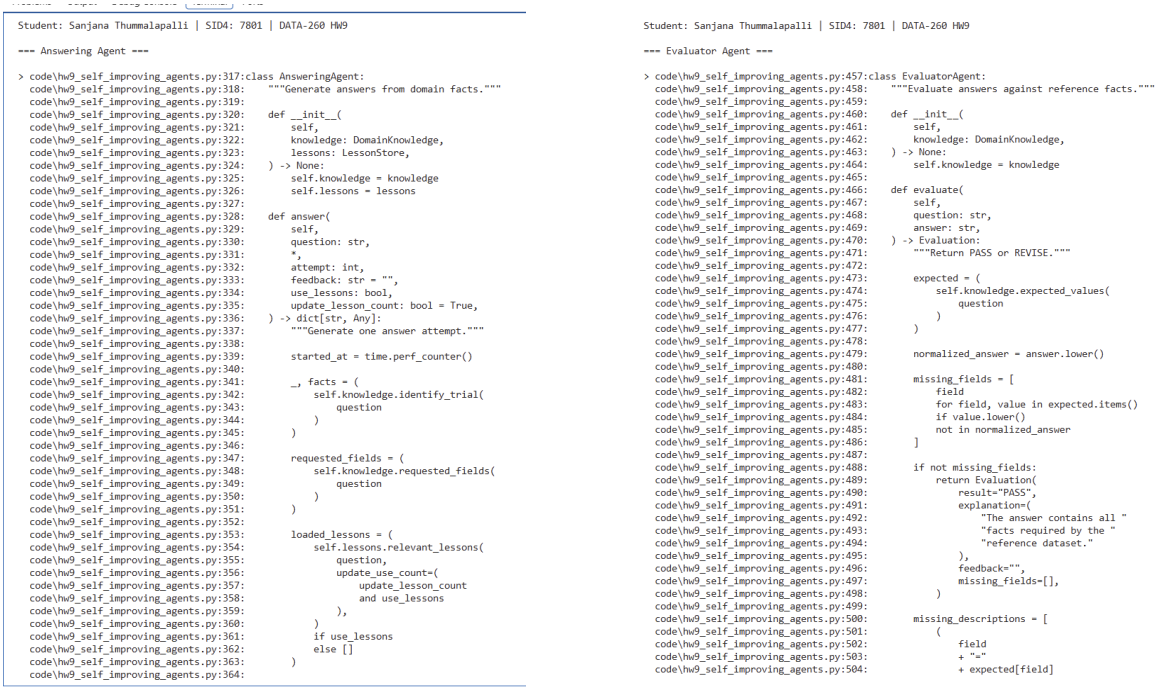
The training, validation and test datasets contain only input questions. Expected answers are not included. The Evaluator Agent derives required facts from reference_data.json.

Agent Communication

The Answering Agent receives a question and generates an answer. The Evaluator Agent compares the answer with the relevant reference facts and returns:

- PASS or REVISE
- An explanation
- Corrective feedback
- Missing fields

When the evaluator returns REVISE, the feedback is sent back to the Answering Agent. The Answering Agent produces one revised response. The system stops after a PASS or after two attempts.



```
Student: Sanjana Thummalapalli | SID4: 7801 | DATA-260 HW9

=== Answering Agent ===

> code\hw9_self_improving_agents.py:317:class AnsweringAgent:
  code\hw9_self_improving_agents.py:318:    """Generate answers from domain facts."""
  code\hw9_self_improving_agents.py:319:
  code\hw9_self_improving_agents.py:320:    def __init__(
  code\hw9_self_improving_agents.py:321:        self,
  code\hw9_self_improving_agents.py:322:        knowledge: DomainKnowledge,
  code\hw9_self_improving_agents.py:323:        lessons: LessonStore,
  code\hw9_self_improving_agents.py:324:    ) -> None:
  code\hw9_self_improving_agents.py:325:        self.knowledge = knowledge
  code\hw9_self_improving_agents.py:326:        self.lessons = lessons
  code\hw9_self_improving_agents.py:327:
  code\hw9_self_improving_agents.py:328:    def answer(
  code\hw9_self_improving_agents.py:329:        self,
  code\hw9_self_improving_agents.py:330:        question: str,
  code\hw9_self_improving_agents.py:331:        *,
  code\hw9_self_improving_agents.py:332:        attempt: int,
  code\hw9_self_improving_agents.py:333:        feedback: str = "",
  code\hw9_self_improving_agents.py:334:        use_lessons: bool,
  code\hw9_self_improving_agents.py:335:        update_lesson_count: bool = True,
  code\hw9_self_improving_agents.py:336:    ) -> dict[str, Any]:
  code\hw9_self_improving_agents.py:337:        """Generate one answer attempt."""
  code\hw9_self_improving_agents.py:338:
  code\hw9_self_improving_agents.py:339:        started_at = time.perf_counter()
  code\hw9_self_improving_agents.py:340:
  code\hw9_self_improving_agents.py:341:        _, facts = (
  code\hw9_self_improving_agents.py:342:            self.knowledge.identify_trial(
  code\hw9_self_improving_agents.py:343:                question
  code\hw9_self_improving_agents.py:344:            )
  code\hw9_self_improving_agents.py:345:        )
  code\hw9_self_improving_agents.py:346:
  code\hw9_self_improving_agents.py:347:        requested_fields = (
  code\hw9_self_improving_agents.py:348:            self.knowledge.requested_fields(
  code\hw9_self_improving_agents.py:349:                question
  code\hw9_self_improving_agents.py:350:            )
  code\hw9_self_improving_agents.py:351:        )
  code\hw9_self_improving_agents.py:352:
  code\hw9_self_improving_agents.py:353:        loaded_lessons = (
  code\hw9_self_improving_agents.py:354:            self.lessons.relevant_lessons(
  code\hw9_self_improving_agents.py:355:                question,
  code\hw9_self_improving_agents.py:356:                update_use_count=(
  code\hw9_self_improving_agents.py:357:                    update_lesson_count
  code\hw9_self_improving_agents.py:358:                    and use_lessons
  code\hw9_self_improving_agents.py:359:                ),
  code\hw9_self_improving_agents.py:360:            )
  code\hw9_self_improving_agents.py:361:            if use_lessons
  code\hw9_self_improving_agents.py:362:            else []
  code\hw9_self_improving_agents.py:363:        )
  code\hw9_self_improving_agents.py:364:
```

```
Student: Sanjana Thummalapalli | SID4: 7801 | DATA-260 HW9

=== Evaluator Agent ===

> code\hw9_self_improving_agents.py:457:class EvaluatorAgent:
  code\hw9_self_improving_agents.py:458:    """Evaluate answers against reference facts."""
  code\hw9_self_improving_agents.py:459:
  code\hw9_self_improving_agents.py:460:    def __init__(
  code\hw9_self_improving_agents.py:461:        self,
  code\hw9_self_improving_agents.py:462:        knowledge: DomainKnowledge,
  code\hw9_self_improving_agents.py:463:    ) -> None:
  code\hw9_self_improving_agents.py:464:        self.knowledge = knowledge
  code\hw9_self_improving_agents.py:465:
  code\hw9_self_improving_agents.py:466:    def evaluate(
  code\hw9_self_improving_agents.py:467:        self,
  code\hw9_self_improving_agents.py:468:        question: str,
  code\hw9_self_improving_agents.py:469:        answer: str,
  code\hw9_self_improving_agents.py:470:    ) -> Evaluation:
  code\hw9_self_improving_agents.py:471:        """Return PASS or REVISE."""
  code\hw9_self_improving_agents.py:472:
  code\hw9_self_improving_agents.py:473:        expected = (
  code\hw9_self_improving_agents.py:474:            self.knowledge.expected_values(
  code\hw9_self_improving_agents.py:475:                question
  code\hw9_self_improving_agents.py:476:            )
  code\hw9_self_improving_agents.py:477:        )
  code\hw9_self_improving_agents.py:478:
  code\hw9_self_improving_agents.py:479:        normalized_answer = answer.lower()
  code\hw9_self_improving_agents.py:480:
  code\hw9_self_improving_agents.py:481:        missing_fields = [
  code\hw9_self_improving_agents.py:482:            field
  code\hw9_self_improving_agents.py:483:            for field, value in expected.items()
  code\hw9_self_improving_agents.py:484:            if value.lower()
  code\hw9_self_improving_agents.py:485:            not in normalized_answer
  code\hw9_self_improving_agents.py:486:        ]
  code\hw9_self_improving_agents.py:487:
  code\hw9_self_improving_agents.py:488:        if not missing_fields:
  code\hw9_self_improving_agents.py:489:            return Evaluation(
  code\hw9_self_improving_agents.py:490:                result="PASS",
  code\hw9_self_improving_agents.py:491:                explanation=(
  code\hw9_self_improving_agents.py:492:                    "The answer contains all "
  code\hw9_self_improving_agents.py:493:                    "facts required by the "
  code\hw9_self_improving_agents.py:494:                    "reference dataset."
  code\hw9_self_improving_agents.py:495:                ),
  code\hw9_self_improving_agents.py:496:                feedback="",
  code\hw9_self_improving_agents.py:497:                missing_fields=[],
  code\hw9_self_improving_agents.py:498:            )
  code\hw9_self_improving_agents.py:499:
  code\hw9_self_improving_agents.py:500:        missing_descriptions = [
  code\hw9_self_improving_agents.py:501:            (
  code\hw9_self_improving_agents.py:502:                field
  code\hw9_self_improving_agents.py:503:                + "="
  code\hw9_self_improving_agents.py:504:                + expected[field]
```

Figure 11: Answering Agent and Evaluator Agent code

4.1 Complete Revision Sequence

Question: Give the phase and enrollment target of the sleep quality trial.

Initial answer: Sleep Quality and Academic Performance Study: Phase: Phase II.

Evaluator result: REVISE

Evaluator feedback:

Add the following missing ground-truth fact

enrollment_target=120.

Revised answer:

Sleep Quality and Academic Performance Study:
Phase: Phase II. Enrollment target: 120.

Final evaluation: PASS

Number of attempts: 2

```
Student: Sanjana Thummalapalli | SID4: 7801 | DATA-260 HW9

=== Evaluator Feedback and Answer Revision ===
Question: Give the phase and enrollment target of the sleep quality trial.

Attempt 1: Sleep Quality and Academic Performance Study: Phase: Phase II.
Evaluation 1: REVISE
Feedback: Add the following missing ground-truth facts: enrollment_target=120.

Attempt 2: Sleep Quality and Academic Performance Study: Phase: Phase II. Enrollment target: 120.
Evaluation 2: PASS
Attempts: 2
Final result: PASS
```

Figure 12: Complete two-attempt revision sequence

4.2 Lesson Creation and Reuse

The same enrollment-omission failure appeared in two training cases. The system created:


```
{
  "lesson_id": "lesson_001",
  "failure_pattern": "The answer omits the requested enrollment target.",
  "correction": "When a question requests both phase and enrollment, or asks for trial details, include the numeric enrollment target.",
  "created_at": "2026-11-09",
  "use_count": 11
}
```


A later question asked for the phase and enrollment target of the digital wellness trial. The Answering Agent loaded lesson_001 and produced a complete answer on the first attempt.

Loaded lessons: ['lesson_001']
Attempts: 1
Evaluation: PASS

```
Student: Sanjana Thummalapalli | SID4: 7801 | DATA-260 HW9

=== Stored Reusable Lesson ===
Lesson ID: lesson_001
Failure pattern: The answer omits the requested enrollment target.
Correction: When a question requests both phase and enrollment, or asks for trial details, include the numeric enrollment target.
Created: 2026-11-09
Use count: 11

=== Later Lesson Reuse ===
Question: Report both the phase and enrollment target for the digital wellness trial.
Loaded lesson: lesson_001
Answer: Digital Wellness Intervention Study: Phase: Phase I. Enrollment target: 180.
Attempts: 1
Evaluation: PASS
```

Figure 13: lessons.json and later lesson-reuse example

4.3 Validation Before Keeping the Lesson

Validation results:

Metric	**Result**
Correct before lesson	4/8
Correct after lesson	8/8
Improved cases	4

Regressions	0
Lesson retained	Yes

The lesson was retained because failure cases improved, previously correct responses remained correct, and unrelated question types did not receive unsupported facts.

The held-out test dataset was not used to create or modify the lesson.

```
Student: Sanjana Thummalapalli | SID4: 7801 | DATA-260 HW9

=== Lesson Validation Gate ===
Correct before lesson: 4/8
Correct after lesson: 8/8
Improved cases: validation_01, validation_02, validation_06, validation_08
Regressions: 0
Lesson kept: True
```

Figure 14: Validation before and after with no regressions

4.4 Four System Variants

The unchanged held-out test dataset was evaluated separately with all four variants.

Variant	**Correct**	**Incorrect**	**Average attempts**	**Average latency**
Baseline	4	4	1.000	0.022 ms
Evaluator Repair	8	0	1.500	0.028 ms
Lesson Memory	8	0	1.000	3.357 ms
Validated Improvement	8	0	1.000	3.460 ms

The baseline failed four questions because it omitted requested enrollment values. Evaluator Repair corrected every failed response but required an average of 1.5 attempts. Lesson Memory and Validated Improvement achieved 8/8 correctness with one attempt per question.

The additional latency for lesson-based variants came from loading and updating persistent lesson memory.

```
Student: Sanjana Thummalapalli | SID4: 7801 | DATA-260 HW9

=== Held-Out Test: Four System Variants ===

variant              correct_answers incorrect_answers average_attempts average_latency_ms
-------              --------------- ----------------- ---------------- ------------------
Baseline             4               4                 1                0.022
Evaluator Repair     8               0                 1.5              0.028
Lesson Memory        8               0                 1                3.357
Validated Improvement 8              0                 1                3.46
```

Figure 15: Four-variant held-out test table

Limitation

The evaluator depends on curated reference facts and keyword-based trial identification. Questions about unseen clinical trials would require additional reference records or a stronger retrieval mechanism. The validation set also represents a limited sample and cannot guarantee that every future lesson will generalize safely.

Automated Verification

The final verification checked:

- Personal configuration
- Redis connectivity
- Redis vector index
- Redis key prefixes
- Database view-count column
- View-counter behavior
- Semantic-cache performance
- Forty labeled cache cases
- Five-threshold sweep
- Complete false-hit evidence
- Stale-cache invalidation
- Required agent datasets
- Evaluator repair
- Lesson reuse
- Four validated variants

Final result:

```
Student: Sanjana Thummalapalli | SID4: 7801 | DATA-260 HW9

=== Homework 9 Automated Verification ===
DATA-260 Homework 9 Verification
PASS | personal_configuration | SID4=7801, PREFIX=s7801, embedding_dimensions=384
PASS | redis_connection | Redis PING succeeded.
PASS | redis_vector_index | Indexes=['s7801:semantic_cache:index']; expected=s7801:semantic_cache:index
PASS | redis_key_prefix | Checked 1 Redis keys; all begin with s7801.
PASS | database_view_column | clinical_trials.view_count exists.
PASS | redis_view_counter | 25 accesses, database syncs at 10 and 20, and PASS found.
PASS | semantic_cache_performance | queries=10, hits=3, hit_rate=30.0%, cached_latency=7.922 ms, miss_latency=7105.463 ms
PASS | labeled_cache_cases | cases=40, categories={'exact_duplicate': 10, 'paraphrase': 10, 'similar_different_answer': 10, 'unrelated': 10}
PASS | threshold_sweep | thresholds=[0.75, 0.8, 0.85, 0.9, 0.95], recommended=0.9
PASS | complete_false_hit | False-hit evidence includes both queries, similarity, cached answer and correct answer.
PASS | stale_cache_invalidation | Stale hit demonstrated, stale entry deleted, and refreshed answer verified.
PASS | agent_datasets | reference_facts=4, training=8, validation=8, test=8, input_only=True
PASS | evaluator_revision | Initial attempt REVISE; second attempt PASS.
PASS | lesson_memory_reuse | lessons=1, use_count=11
PASS | validated_four_variants | variants=4, before=4, after=8, regressions=[]

Passed: 15/15
Overall: PASS
Saved: reports\hw09\verification.json
```

Figure 16: Automated verification showing 15/15 and Overall PASS

AI Use Statement

I used an AI assistant to help interpret the Homework 9 requirements, plan the Redis and agent architecture, draft portions of the implementation, troubleshoot errors, prepare test commands, and organize the report.

I personally installed and configured Redis Stack, ran the Docker services, executed every experiment, inspected Redis and MySQL state, committed the labeled cache cases before the threshold sweep, ran the four agent variants, corrected files, and verified all results.

Some generated instructions initially caused issues. A PowerShell here-string marker was accidentally included in a JSON file, and a verification script assumed UTF-8 even though PowerShell had created a UTF-16 log. I detected these problems through JSON parsing, py_compile, Ruff and runtime tracebacks. I removed the invalid marker and updated the verifier to support both encodings.

The complete AI-use explanation is also stored in reports/hw09/AI_USE.md.

Conclusion

The Homework 9 implementation successfully added Redis caching and self-improving agents to the Clinical Trials system. The view counter performed atomic Redis increments and synchronized to MySQL every ten accesses. The semantic cache used a Redis HNSW vector index and reduced average repeated-query latency from 7105.463 ms to 7.922 ms.

The threshold experiment demonstrated that higher hit rates can introduce incorrect responses. Threshold 0.90 was selected because it produced no false hits and the lowest modeled total cost. Data-version invalidation successfully detected and removed a stale answer.

The Evaluator Agent repaired incomplete responses, reusable lesson memory reduced repeated failures, and validation prevented regressions. The final validated system achieved 8/8 correct held-out answers in one attempt per question. Automated verification passed all 15 checks.